Principaux algorithmes de Machine Learning

Fondements, principes détaillés, notations et applications

Python Support universitaire – Licence avancée et Master

2026

Objectifs et acquis attendus

Choisir un paradigme en fonction d’une problématique réelle.

Comprendre l’intuition, les hypothèses et les équations de 29 méthodes.

Interpréter chaque symbole, paramètre appris et hyperparamètre.

Mettre en œuvre les approches sous Python avec évaluation rigoureuse.

Identifier le surapprentissage, les fuites de données et les limites métier.

Plan général — Partie 1

1. Fondements : données, apprentissage, généralisation et métriques.
2. Régressions : linéaire, pénalisées, polynomiale, arbres et ensembles.
3. Classification : logistique, KNN, Bayes, arbres, SVM, boosting.

Plan général — Partie 2

4. Non supervisé : partitionnement, ACP, projections et anomalies.
5. Renforcement : bandits, exploration, Q-learning, SARSA et DQN.
6. Comparaisons méthodologiques et choix des modèles.

7. Travaux pratiques et exercices ; annexes bibliographiques.

Science des données : démarche

Question économique et indicateur à améliorer

Collecte, qualité, exploration et connaissance métier

Choix d’une base de comparaison avant l’algorithme

Expérimentation, évaluation, déploiement et suivi

Trois grands paradigmes

Supervisé : paires (x,y), prédire une cible observée

Non supervisé : structure de X sans utiliser de cible

Renforcement : agent, état, action, récompense, politique

Forme des données

X : matrice de n lignes et p caractéristiques

y : vecteur cible de longueur n (si supervisé)

Observations indépendantes ou structurées temporellement

Numériques, catégorielles, textuelles ou séquentielles

Paramètre appris et hyperparamètre

Paramètre appris : coefficient beta, centroïde mu ou poids neuronal

Hyperparamètre : alpha de Ridge, K de K-means, profondeur d’arbre

Les hyperparamètres sont choisis via validation, jamais sur le test final

Fonction de coût

La perte mesure le désaccord entre prédiction et vérité

L’optimiseur cherche à minimiser l’erreur et éventuellement une pénalité

Attention au choix de l’unité et de la métrique métier

Descente de gradient

Calculer le gradient par rapport aux paramètres

Faire une mise à jour de pas eta

Réitérer jusqu’à convergence ou nombre maximal d’itérations

Sous-apprentissage

Modèle trop simple : erreurs élevées sur entraînement et validation

Envisager variables plus informatives ou modèle plus flexible

Surapprentissage

Erreur d’entraînement très faible, validation dégradée

Remèdes : validation correcte, régularisation, moins de complexité

Biais–variance

Biais élevé : classe de modèles rigide

Variance élevée : modèle sensible aux perturbations de l’échantillon

Le bruit irréductible limite les performances

Séparation des données

Entraînement : ajustement des paramètres

Validation : réglage des hyperparamètres

Test : évaluation finale et indépendante une seule fois

Validation croisée

K plis : répéter les ajustements sur partitions distinctes

Stratifier si classes déséquilibrées

GroupKFold si observations liées, TimeSeriesSplit pour chronologie

Fuite de données

Ne jamais standardiser ou imputer avec le test

Écarter les prédicteurs indisponibles à la date de décision

Utiliser Pipeline et ColumnTransformer

Standardisation

Mettre les variables sur des échelles comparables

Indispensable pour KNN, SVM, Ridge, PCA ; souvent secondaire pour arbres

Échantillonnage déséquilibré

Ne pas juger uniquement par accuracy

Utiliser précision, rappel, F1, PR-AUC et courbes de calibration

Validation temporelle

Les données futures ne doivent jamais influencer la préparation passée

Utiliser un découpage chronologique ou une validation glissante

Interprétation et prudence

Importance prédictive ne prouve pas une cause

Comparer les groupes, inspecter dérive, documenter les limites

MAE : erreur absolue moyenne

Formule

$$\text{MAE} = \frac{1}{n}\sum_{i}|y_i - \hat{y}_i|$$

Paramètres

n : nombre d'observations ; y_i : valeur vraie ; y-hat_i : prédiction ; |.| : valeur absolue.

Exemple / interprétation

Exemple : vrais (2,4), prédits (3,6) => MAE=1,5.

MSE : erreur quadratique moyenne

Formule

$$MSE = \frac{1}{n}\sum_{i}(y_i - \hat{y}_i)^2$$

Paramètres

n : observations ; y_i : vérité ; y-hat_i : prévision ; carré : fortes erreurs davantage pénalisées.

Exemple / interprétation

Erreurs 1 et 2 => MSE=(1+4)/2=2,5.

RMSE : racine MSE

Paramètres

$\overline{X_i}$

$(y_i - yb_i)^2$

Formule

RMSE =

$s_{1\ n}$

n : observations ; y_i : vérité ; y-hat_i : estimation ; sqrt : racine carrée pour revenir à l'unité de y.

Exemple / interprétation

Pour MSE=2,5, RMSE≈1,58.

R^2 : pouvoir descriptif

Formule

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Paramètres

y-bar : moyenne des valeurs y ; numérateur : somme des carrés des résidus ; dénominateur : variabilité totale.

Exemple / interprétation

$R^2=0$ signifie performance égale à la moyenne de référence sur ce jeu.

Précision, rappel, F1

Formule

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}$$

Paramètres

TP : vrais positifs ; FP : faux positifs ; FN : faux négatifs ; P : précision ; R : rappel ; F1 : moyenne harmonique.

Exemple / interprétation

TP=8, FP=2, FN=4 : précision=0,8 ; rappel≈0,667 ; F1≈0,727.

Exactitude et matrice de confusion

Formule

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Paramètres

TP,TN : bonnes prédictions positive/négative ; FP,FN : erreurs ; dénominateur : total.

Exemple / interprétation

Sur 100 dont 95 négatifs, prédire toujours négatif donne 95 % sans détecter

un positif.

Formule

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}$$

Paramètres

TPR : rappel ; FPR : taux de faux positifs ; ROC : TPR contre FPR selon le seuil ; PR : précision contre rappel.

Exemple / interprétation

Lorsque positifs rares, la PR-AUC reflète mieux la qualité des alertes.

Silhouette et clustering

Formule

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Paramètres

a(i) : distance moyenne aux points du groupe i ; b(i) : meilleure distance moyenne aux autres groupes ; s(i) : coefficient de silhouette.

Exemple / interprétation

Si a=2 et b=6 : silhouette=4/6≈0,667.

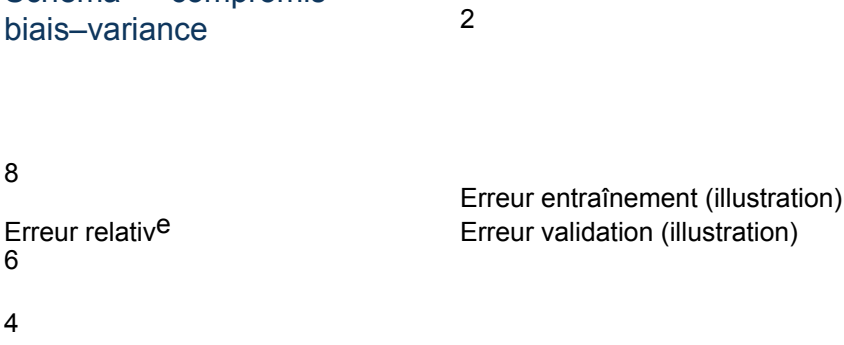
Schéma — protocole sans fuite

Données originales

Entraînement 70% Validation 15% Test 15%

Les proportions sont illustratives ; le schéma s’adapte aux données temporelles.

Schéma — compromis biais–variance



0

0 1 2 3 4 5 6 7 8 9 10 Complexité du modèle

Expliquer le principe opérationnel des 29 algorithmes, étape par étape.

Relier intuition, fonctionnement, équations et paramètres.

Calcul numérique, schéma et exemple Python.

Discussion : limites, métriques et applications.

1. Régression linéaire — intuition

Définition

Ajuster une droite ou un hyperplan qui minimise les erreurs quadratiques.

Problème étudié : Prévision du chiffre d’affaires

Famille : régression supervisée

On distingue les entrées, la règle apprise et la sortie attendue.

1. Régression linéaire — principe de fonctionnement

Principe : Régression ordinaire par moindres carrés

Construire la matrice des caractéristiques et ajouter une constante.

Prédire une valeur par une combinaison pondérée des caractéristiques.
Chercher les coefficients qui minimisent la somme des carrés des erreurs.
Utiliser les coefficients estimés pour prédire de nouvelles observations.

Entrées Règle / optimisation Modèle appris Sortie / action

Point de vigilance

Les coefficients décrivent des variations conditionnelles, sans établir de causalité.

1. Régression linéaire — étapes

Données / états Prétraitement Algorithme Résultat / métrique

À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Ajuster une droite ou un hyperplan qui minimise les erreurs quadratiques.

1. Régression linéaire — équation

pédagogique

Modèle / critère principal

$$\hat{y}_i = \beta_0 + \sum_{j=1}^{p} \beta_j x_{ij}\,,\quad J = \frac{1}{2n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Lecture

Ajuster une droite ou un hyperplan qui minimise les erreurs quadratiques.

1. Régression linéaire — notations

Tous les symboles de l’équation

n : nombre de lignes ; p : nombre de caractéristiques ; x_{ij} : variable j du client i ; y_i : vraie valeur

; beta_0 : constante ; beta_j : coefficient estimé ; J : coût moyen quadratique.

Paramètres et hyperparamètres

Hyperparamètre principal : fit_intercept. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

1. Régression linéaire — exemple calculé

Exemple numérique

Avec beta_0=2, beta_1=3 et x=4 : prediction=14. Si y=15, erreur=1.

Question de compréhension

Quelle modification de fit_intercept pourrait changer la complexité, la stabilité

ou l’exploration ? Justifier.

1. Régression linéaire — mise en pratique

Python — API ou principe

LinearRegression()

Avantage : Rapide, explicable

Limite : Sensibilité aux valeurs atypiques et à la non-linéarité

Travail : entraîner, comparer une référence et visualiser une métrique.

Code complet associé : notebook « 29 exemples algorithmes ».

2. Régression Ridge — intuition

Définition

Régulariser les coefficients par une pénalité quadratique.

Problème étudié : Prévision avec variables économiques corrélées

Famille : régression supervisée

On distingue les entrées, la règle apprise et la sortie attendue.

2. Régression Ridge — principe de fonctionnement

Principe : Régularisation L2

Standardiser les variables pour rendre la pénalité comparable.

Calculer les prédictions du modèle linéaire.

Minimiser erreur quadratique et pénalité L2 sur les coefficients.

Choisir la force de régularisation par validation croisée.

Point de vigilance

La pénalité rapproche les coefficients de zéro sans les annuler généralement.

2. Régression Ridge — étapes

À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante.
Régulariser les coefficients par une pénalité quadratique.

2. Régression Ridge — équation

Modèle / critère principal

$$J(\beta) = \frac{1}{n} \| y - X\beta \|_2^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

Lecture pédagogique

Régulariser les coefficients par une pénalité quadratique.

2. Régression Ridge — notations

Tous les symboles de l’équation

X : matrice n x p ; y : vecteur n x 1 ; beta : vecteur p x 1 (interception exclue de la pénalité) ; lambda : force de régularisation ; ||.||_2 : norme euclidienne.

Paramètres et hyperparamètres

Hyperparamètre principal : alpha. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

2. Régression Ridge — exemple calculé

Exemple numérique

Si beta=(2,1) et lambda=0,5, la pénalité vaut 0,5(4+1)=2,5.

Question de compréhension

Quelle modification de alpha pourrait changer la complexité, la stabilité ou

l’exploration ? Justifier.

2. Régression Ridge — mise en pratique

Python — API ou principe

Ridge(alpha=1.0)

Avantage : Stable avec colinéarité

Limite : Ne sélectionne pas automatiquement des variables

Travail : entraîner, comparer une référence et visualiser une métrique.

Code complet associé : notebook « 29 exemples algorithmes ».

3. Régression Lasso — intuition

Définition

Favoriser une solution parcimonieuse avec pénalité absolue.

Problème étudié : Sélection de ratios financiers

Famille : régression supervisée

On distingue les entrées, la règle apprise et la sortie attendue.

3. Régression Lasso — principe de fonctionnement

Principe : Régularisation L1

Centrer et mettre les caractéristiques à la même échelle.

Produire des prédictions linéaires.

Minimiser erreur quadratique et somme des valeurs absolues des coefficients. Sélectionner la pénalité sur validation et examiner les coefficients nuls.

Entrées Règle / optimisation Modèle appris Sortie / action

Point de vigilance

L1 permet une sélection de variables, instable en présence de forts liens entre variables.

3. Régression Lasso — étapes

Données / états Prétraitement Algorithme Résultat / métrique

À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Favoriser une solution parcimonieuse avec pénalité absolue.

3. Régression Lasso — équation

Modèle / critère principal

$$J(\beta) = \frac{1}{2n} \| y - X\beta \|_2^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

Lecture pédagogique

Favoriser une solution parcimonieuse avec pénalité absolue.

3. Régression Lasso — notations

Tous les symboles de l’équation

n : taille de l’échantillon ; X : matrice explicative ; y : cible ; beta_j : coefficients ; lambda : pénalité L1 ; |.| : valeur absolue.

Paramètres et hyperparamètres

Hyperparamètre principal : alpha. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

3. Régression Lasso — exemple calculé

Exemple numérique

Si beta=(2,-1) et lambda=0,5, la pénalité vaut 1,5.

Question de compréhension

Quelle modification de alpha pourrait changer la complexité, la stabilité ou

l'exploration ? Justifier.

3. Régression Lasso — mise en pratique

Python — API ou principe

```
Lasso(alpha=0.1, max_iter=10000)
```

Avantage : Sélection intégrée

Limite : Instable en présence de variables très corrélées

Travail : entraîner, comparer une référence et visualiser une métrique.

Code complet associé : notebook « 29 exemples algorithmes ».

4. Régression polynomiale — intuition

Définition

Ajouter des puissances et interactions pour modéliser une courbe.

Problème étudié : Évolution non linéaire des prix

Famille : régression supervisée

On distingue les entrées, la règle apprise et la sortie attendue.

4. Régression polynomiale — principe de fonctionnement

Principe : Transformation des caractéristiques

Créer des puissances et interactions des caractéristiques.

Appliquer un modèle linéaire à cet espace enrichi.

Estimer ses coefficients sur le jeu d’entraînement.

Choisir le degré sur validation et contrôler le surapprentissage.

Entrées Règle / optimisation Modèle appris Sortie / action

Point de vigilance

Polynomiale dans les variables ne veut pas dire non linéaire dans les coefficients.

4. Régression polynomiale — étapes

Données / états Prétraitement Algorithme Résultat / métrique

À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Ajouter des puissances et interactions pour modéliser une courbe.

4. Régression polynomiale — équation

Modèle / critère principal

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d$$

Lecture pédagogique

Ajouter des puissances et interactions pour modéliser une courbe.

Tous les symboles de l’équation

x : variable d’entrée ; d : degré maximal ; beta_j : paramètres appris ; y-hat : valeur estimée.

Paramètres et hyperparamètres

Hyperparamètre principal : degree. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

4. Régression polynomiale — exemple calculé

Exemple numérique

Avec beta=(1,2,0,5) et x=2 : prediction=1+4+2=7.

Question de compréhension

Quelle modification de degree pourrait changer la complexité, la stabilité ou

l’exploration ? Justifier.

4. Régression polynomiale — mise en pratique

Python — API ou principe

```
make_pipeline(PolynomialFeatures(2), LinearRegression())
```

Avantage : Modélise courbes simples

Limite : Explose en dimension et peut surapprendre

Travail : entraîner, comparer une référence et visualiser une métrique.

Code complet associé : notebook « 29 exemples algorithmes ».

5. Arbre de régression — intuition

Définition

Découper l’espace en feuilles et prédire leur moyenne.

Problème étudié : Estimation des coûts logistiques

Famille : régression supervisée

On distingue les entrées, la règle apprise et la sortie attendue.

5. Arbre de régression — principe de fonctionnement

Principe : Découpage récursif

Examiner plusieurs variables et seuils de coupure.

Retenir la séparation qui réduit le plus la dispersion des cibles.

Répéter récursivement dans les sous-ensembles.

Prédire la moyenne des cibles dans la feuille atteinte.

Entrées Règle / optimisation Modèle appris Sortie / action

Point de vigilance

Profondeur et taille minimale des feuilles contrôlent la complexité.

5. Arbre de régression — étapes

Données / états Prétraitement Algorithme Résultat / métrique

À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Découper l’espace en feuilles et prédire leur moyenne.

5. Arbre de régression — équation

Modèle / critère principal

$$\hat{y}_b(x) = \frac{1}{|L(x)|} \sum_{i \in L(x)} y_i$$

Lecture pédagogique

Découper l’espace en feuilles et prédire leur moyenne.

5. Arbre de régression — notations

Tous les symboles de l’équation

L(x) : indices de la feuille contenant x ; |L(x)| : effectif ; y_i : valeur des observations dans la feuille.

Paramètres et hyperparamètres

Hyperparamètre principal : max_depth. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

5. Arbre de régression — exemple calculé

Exemple numérique

Dans une feuille contenant 8, 10 et 12, la prédiction vaut 10.

Question de compréhension

Quelle modification de max_depth pourrait changer la complexité, la stabilité

ou l’exploration ? Justifier.

5. Arbre de régression — mise en pratique

Python — API ou principe

DecisionTreeRegressor(max_depth=5, random_state=42)

Avantage : Non linéaire et interprétable

Limite : Instable et extrapole mal

Travail : entraîner, comparer une référence et visualiser une métrique.

Code complet associé : notebook « 29 exemples algorithmes ».

6. Forêt aléatoire — régression — intuition

Définition

Agréger les prédictions d’arbres entraînés sur des échantillons bootstrap.

Problème étudié : Prédiction de la demande export

Famille : régression supervisée

On distingue les entrées, la règle apprise et la sortie attendue.

6. Forêt aléatoire — régression — principe de fonctionnement

Principe : Bagging et agrégation

Tirer plusieurs échantillons bootstrap des observations.

Entraîner des arbres de régression avec des sous-ensembles aléatoires de variables. Calculer la prédiction de chaque arbre sur le nouveau point. Moyenner les prédictions pour réduire la variance.

Entrées Règle / optimisation Modèle appris Sortie / action

Point de vigilance

Les arbres sont entraînés en parallèle, sans correction séquentielle des erreurs.

6. Forêt aléatoire — régression — étapes

Données / états Prétraitement Algorithme Résultat / métrique

À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Agréger les prédictions d'arbres entraînés sur des échantillons bootstrap.

6. Forêt aléatoire — régression — équation

Modèle / critère principal

$$\hat{y}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$$

Lecture pédagogique

$T_b(x)$

Agréger les prédictions d’arbres entraînés sur des échantillons bootstrap.

6. Forêt aléatoire — régression — notations

Tous les symboles de l’équation

B : nombre d’arbres ; T_b : prédiction du b-ième arbre ; x : nouveau point ; y-hat : moyenne.

Paramètres et hyperparamètres

Hyperparamètre principal : n_estimators / max_depth. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

6. Forêt aléatoire — régression — exemple calculé

Exemple numérique

Avec trois arbres donnant 9, 12 et 15 : moyenne=12.

Question de compréhension

Quelle modification de n_estimators / max_depth pourrait changer la complexité, la stabilité

ou l’exploration ? Justifier.

6. Forêt aléatoire — régression — mise en pratique

Python — API ou principe

```
RandomForestRegressor(n_estimators=80, random_state=42)
```

Avantage : Robuste et non linéaire

Limite : Modèle volumineux, extrapolation faible

Travail : entraîner, comparer une référence et visualiser une métrique.

Code complet associé : notebook « 29 exemples algorithmes ».

7. Gradient Boosting — régression — intuition

Définition

Construire des arbres qui corrigent progressivement les résidus.

Problème étudié : Prévision de ventes

Famille : régression supervisée

On distingue les entrées, la règle apprise et la sortie attendue.

7. Gradient Boosting — régression — principe de fonctionnement

Principe : Ensemble séquentiel additif

Initialiser une prédiction simple, souvent la moyenne de la cible.

Calculer les pseudo-résidus de la fonction de perte.

Entraîner un petit arbre à approximer ces corrections.

Ajouter l’arbre avec un taux d’apprentissage ; répéter.

Point de vigilance

Le nombre d’arbres et le taux d’apprentissage déterminent ensemble la capacité du modèle.

7. Gradient Boosting — régression — étapes

À retenir

Choisir les données disponibles à la décision ; conserver une évaluation indépendante. Construire des arbres qui corrigent progressivement les résidus.

7. Gradient Boosting — régression — équation

Modèle / critère principal

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

Lecture pédagogique

Construire des arbres qui corrigent progressivement les résidus.

7. Gradient Boosting — régression — notations

Tous les symboles de l’équation

F_m : modèle à l’itération m ; h_m : nouvel apprenant faible ; eta : taux d’apprentissage ; x : données.

Paramètres et hyperparamètres

Hyperparamètre principal : learning_rate / max_iter. Les coefficients, centroïdes, partitions ou valeurs Q sont appris à partir des données selon le modèle.

7. Gradient Boosting — régression — exemple calculé

Exemple numérique

Prédiction initiale 10, correction +4 avec eta=0,1 : nouveau score=10,4.

Question de compréhension

Quelle modification de learning_rate / max_iter pourrait changer la complexité, la stabilité ou l'exploration

? Justifier.

7. Gradient Boosting — régression — mise en pratique

Python — API ou principe

```
HistGradientBoostingRegressor(max_iter=80, random_state=42)
```

Avantage : Puissant sur tableaux

Limite : Réglage sensible, risque de surapprentissage

Travail : entraîner, comparer une référence et visualiser une métrique.

Code complet associé : notebook « 29 exemples algorithmes ».